

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки
09.03.01 Информатика и вычислительная техника

направленность (профиль)
«Технологии разработки программного обеспечения и обработки больших
данных»

Курсовая работа
по дисциплине «Прикладные информационные технологии»

«Разработка и контейнеризация веб-приложения для отслеживания курсов валют с сайта Центрального банка Российской Федерации с использованием Docker и автоматизация выпуска сертификатов Let's Encrypt»

Обучающейся 3 курса
очной формы обучения
Ивановой Ольги Дмитриевны

Руководитель курсовой работы:
старший преподаватель кафедры ИТЭО
Аксютин Павел Александрович

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	4
1.1. DOCKER. DOCKER COMPOSE.....	4
1.2. NGINX.....	7
1.3. LET’S ENCRYPT	7
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ.	8
2.1. ОПИСАНИЕ ПРИЛОЖЕНИЯ.	8
2.2 ХОД РАБОТЫ.	11
ЗАКЛЮЧЕНИЕ	21
СПИСОК ЛИТЕРАТУРЫ	22

ВВЕДЕНИЕ

В настоящее время веб-приложения являются одним из основных способов предоставления информации и сервисов пользователям. При этом важную роль играет не только разработка самого приложения, но и его развертывание, обеспечение безопасности передачи данных и возможность быстрого обновления программного обеспечения.

Одним из наиболее популярных инструментов контейнеризации является Docker, который позволяет упаковать приложение вместе со всеми необходимыми зависимостями в изолированный контейнер и обеспечить одинаковую среду выполнения на различных вычислительных системах. Использование Docker упрощает развертывание приложений, повышает переносимость программного обеспечения и облегчает его сопровождение.

При публикации веб-приложений в сети Интернет особое значение имеет защита передаваемых данных. Для этого применяется протокол HTTPS, который требует использования SSL/TLS-сертификатов. Бесплатные сертификаты, выпускаемые сервисом Let's Encrypt, получили широкое распространение благодаря возможности автоматического получения и продления, что значительно упрощает администрирование веб-сервисов.

Цель работы: Разработать и выполнить контейнеризацию веб-приложения для отслеживания курсов валют с сайта Центрального банка Российской Федерации с использованием Docker, а также обеспечить защищённый доступ к приложению посредством автоматического выпуска сертификатов Let's Encrypt.

Предмет исследования: Средства контейнеризации Docker, технология оркестрации контейнеров Docker Compose, веб-сервер Nginx, а также механизм автоматического выпуска и обновления SSL-сертификатов Let's Encrypt.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.

1.1. DOCKER. DOCKER COMPOSE.

Деплой (развертывание) — это сложный и многоступенчатый процесс, который обычно включает в себя подготовку окружения, установку зависимостей, миграцию базы данных, минимизацию и компиляцию ассетов, проверку работоспособности и, наконец, переключение трафика на новую версию приложения. Важность этого процесса не может быть недооценена: от качества и эффективности деплоя зависит стабильность работы приложения, удовлетворенность пользователей и, в конечном итоге, успех всего проекта [1].

В современном мире стремительно развивающихся технологий перед разработчиками и системными администраторами стоит общая задача: как эффективно развертывать приложения, управлять ими и масштабировать их в различных средах. Эту задачу решают два основных подхода: виртуальные машины (VM) и контейнеризация (например, Docker). Понимание разницы между этими технологиями имеет решающее значение для принятия взвешенных архитектурных решений. Виртуальные машины десятилетиями были традиционным подходом, но контейнеризация с помощью Docker изменила наше представление о развертывании приложений и использовании ресурсов [2].

Виртуальные машины (VM) — это широко распространенная технология, которая позволяет создавать полноценные изолированные вычислительные среды путём виртуализации аппаратных компонентов (с использованием аппаратной поддержки со стороны процессора). Каждая виртуальная машина работает под управлением собственной операционной системы (гостевой ОС) поверх ОС хоста, а гипервизор управляет распределением ресурсов и их изоляцией. Такая комплексная виртуализация позволяет одновременно запускать несколько операционных систем

(например, Windows и Linux) на одном физическом компьютере, при этом каждая из них считает, что имеет эксклюзивный доступ к выделенным аппаратным ресурсам. Использование нескольких ВМ на одной физической машине позволяет добиться значительной экономии эксплуатационных и энергетических затрат [2],[3].

Однако контейнерная технология, как показывает практика, дает еще более значительные преимущества. В отличие от виртуальных машин, контейнеры не содержат копию операционной системы. По своей сути контейнеризация использует виртуализацию на уровне ОС для создания изолированных пользовательских пространств. Контейнеры используют ядро операционной системы хоста, сохраняя при этом полную изоляцию процессов приложений, файловых систем и сетевых конфигураций. Вместо установки отдельной ОС, на хост-системе устанавливается среда выполнения контейнера, которая становится каналом для всех контейнеров на вычислительной системе [4].

Такой подход делает контейнеры значительно более ресурсоэффективными по сравнению с виртуальными машинами. Контейнер создает единый исполняемый пакет, который объединяет код приложения со всеми сопутствующими конфигурационными файлами, библиотеками и зависимостями, необходимыми для его работы, но без "тяжеловесной" копии гостевой ОС. Это позволяет запускать контейнеры за секунды и обслуживать десятки таких изолированных сред на одном сервере [2][3].

Исходя из сравнения, для реализации поставленных в курсовой работе задач был выбран именно Docker по следующим причинам:

1. Стабильное окружение. Docker гарантирует, что приложение будет работать одинаково везде — на ноутбуке разработчика, тестовом сервере или в продакшене. Всё, что нужно для запуска, уже собрано в контейнер, поэтому код не зависит от различий в системах и настройках.

2. Лёгкость и скорость. Контейнеры используют общее ядро ОС и не создают отдельную копию системы, благодаря чему занимают меньше места и запускаются за секунды, что кардинально отличается от тяжеловесных виртуальных машин.

3. Масштабирование. При росте нагрузки можно просто запустить дополнительные контейнеры — система быстро увеличит мощность и перераспределит ресурсы без изменения кода.

4. Изоляция процессов. Каждый контейнер работает независимо от других. Если в одном произойдёт сбой или утечка памяти, это не затронет другие контейнеры и систему в целом, обеспечивая надежность, сравнимую с ВМ, но с меньшими накладными расходами.

5. Удобная интеграция в CI/CD. Контейнеры уже стали стандартом в современных пайплайнах. Приложение можно собрать, протестировать и запустить в одинаковой среде — от локальной машины до продакшена. Благодаря этому меньше неожиданных багов, а релизы проходят быстрее и спокойнее [5].

Docker Compose — это инструмент для определения и запуска многоконтейнерных приложений. Он позволяет упростить и повысить эффективность разработки и развертывания. Compose упрощает управление всем стеком приложений, позволяя легко управлять сервисами, сетями и томами с помощью одного файла конфигурации YAML. Затем с помощью одной команды вы создаете и запускаете все сервисы из файла конфигурации [6].

С помощью Docker Compose вы используете файл конфигурации YAML, известный как файл Compose, для настройки сервисов вашего приложения, а затем создаете и запускаете все сервисы из своей конфигурации с помощью интерфейса командной строки Compose [7].

1.2. NGINX

Nginx (Engine X, или «Энджин-икс») — это программное обеспечение с открытым исходным кодом для создания веб-серверов. Одной из его основных возможностей является работа в режиме обратного прокси-сервера (Reverse Proxy). В этом случае Nginx принимает HTTP- или HTTPS-запросы от клиентов, перенаправляет их внутреннему приложению, получает ответ и возвращает его пользователю. Такой подход позволяет скрыть внутреннюю структуру приложения, повысить безопасность и упростить обработку сетевых запросов [8], [9].

Использование Nginx в качестве обратного прокси особенно удобно при контейнеризации приложений, поскольку позволяет организовать единый входной узел для нескольких сервисов и выполнять маршрутизацию запросов между ними. Кроме того, Nginx поддерживает работу с SSL/TLS-сертификатами и обеспечивает защищённый доступ к веб-приложениям по протоколу HTTPS [9].

1.3. LET'S ENCRYPT

Let's Encrypt — центр сертификации, предоставляющий бесплатные криптографические сертификаты X.509 для шифрования передаваемых через интернет данных HTTPS и других протоколов, используемых серверами в Интернете. Процесс выдачи сертификатов полностью автоматизирован [10].

Проект Let's Encrypt создан для того, чтобы большая часть интернет-сайтов смогла перейти к шифрованным подключениям (HTTPS). В отличие от коммерческих центров сертификации, в данном проекте не требуется оплата, переконфигурация веб-серверов, использование электронной почты, обработка просроченных сертификатов, что делает процесс установки и настройки TLS-шифрования значительно более простым [10].

Срок действия сертификатов составляет 90 дней, и их необходимо регулярно продлевать. Процесс продления сертификата также можно автоматизировать благодаря протоколу ACME. Процесс обновления сертификата можно легко настроить с помощью различных инструментов и клиентов командной строки (например, Certbot). Благодаря автоматическому обновлению веб-сайты продолжают работать безопасно и бесперебойно [11].

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ.

2.1. ОПИСАНИЕ ПРИЛОЖЕНИЯ.

В рамках курсовой работы используется готовое веб-приложение для отслеживания курсов валют Центрального банка Российской Федерации. Приложение позволяет получать актуальные курсы валют с официального API ЦБ РФ и отображать их пользователю через веб-интерфейс.

Приложение реализовано в соответствии с архитектурным шаблоном MVC (Model-View-Controller). Модель отвечает за получение и хранение данных, контроллер обрабатывает запросы пользователя и взаимодействует с моделью, а представление отображает информацию в браузере с использованием HTML-шаблонов.

Функциональные возможности приложения:

- a. Получение актуальных курсов валют с сайта ЦБ РФ;
- b. выбор пользователем отслеживаемых валют;
- c. отображение списка валют, времени обновления и текущего курса;
- d. обработка ошибок при недоступности внешнего сервиса или некорректном вводе данных.

При разработке приложения использовались следующие программные средства:

1. Язык программирования — Python 3;
2. веб-фреймворк — Flask;
3. ORM — SQLAlchemy;
4. система управления базами данных — SQLite;
5. шаблонизатор — Jinja2;
6. библиотека для HTTP-запросов — Requests;
7. веб-сервер приложений — Gunicorn;
8. система контейнеризации — Docker;
9. средство оркестрации контейнеров — Docker Compose;
10. веб-сервер и обратный прокси — Nginx.

Приложение состоит из следующих основных файлов:

1. app.py — контроллер приложения, реализующий обработку HTTP-запросов и взаимодействие между моделью и представлением;
2. model.py — модель приложения, содержащая описание таблицы базы данных и класс для получения курсов валют;
3. templates/index.html — HTML-шаблон, отвечающий за отображение пользовательского интерфейса;

Системные требования.

ОС:

- a. Windows 10/11;
- b. Ubuntu 24.04 LTS или более новая версия Linux.

ПО:

- a. Python версии 3.13;
- b. Docker версии 20 и выше;
- c. Docker Compose версии 2 и выше;
- d. Nginx;

- e. Certbot для автоматического выпуска и обновления сертификатов Let's Encrypt.

Используемая база данных: SQLite 3.

Используемые библиотеки Python:

- a. Flask;
- b. SQLAlchemy;
- c. Requests;
- d. Gunicorn.

Ограничения приложения:

- a. Получение данных возможно только при наличии подключения к сети Интернет;
- b. поддерживаются только валюты, присутствующие в ответе API ЦБ РФ;
- c. база данных SQLite предназначена для небольших объёмов данных и не рассчитана на большое количество одновременных пользователей;
- d. пользовательский интерфейс реализован в виде одной веб-страницы без системы авторизации и разграничения прав доступа.

2.2 ХОД РАБОТЫ.

Начнем с создания поддомена (Рисунок 1).

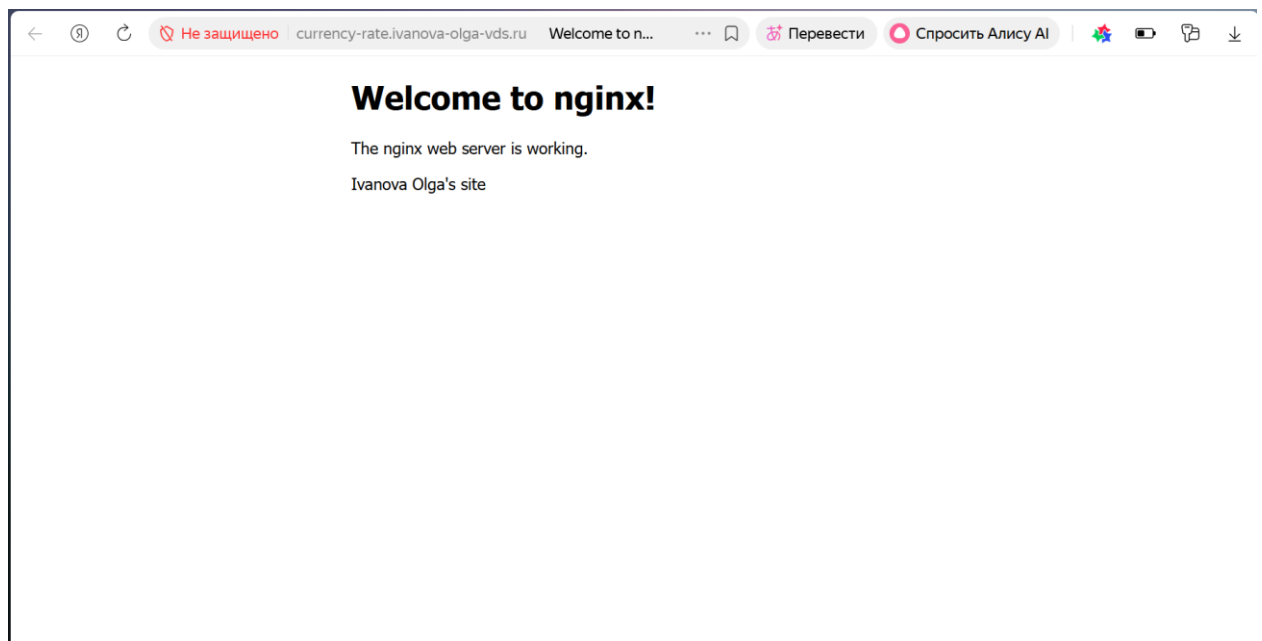


Рисунок 1 – Демонстрация поддомена <http://currency-rate.ivanova-olga-vds.ru>

Листинг 1 - Создание рабочей директории проекта

```
mkdir /root/currency-rate  
cd /root/currency-rate
```

Установка зависимостей (Рисунок 2).

```
root@olya-vds:~/currency-rate# cat requirements.txt
flask
sqlalchemy
requests
gunicorn
```

Рисунок 2 – requirements.txt

Листинг 2 - Создание Dockerfile

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]
```

Листинг 3 - Создание docker-compose.yml

```
services:

  app:

    build: .

    container_name: currency-app

    ports:

      - "5003:5000"
```

```
restart: always
```

Демонстрация файлов проекта, размещённых на сервере (Рисунок 3).

```
root@olya-vds:~/currency-rate# ls -la
total 32
drwxr-xr-x  3 root root 4096 Jun 23 11:46 .
drwx----- 11 root root 4096 Jun 23 10:38 ..
-rw-r--r--  1 root root  187 Jun 23 10:43 Dockerfile
-rw-r--r--  1 root root 1479 Jun 23 11:46 app.py
-rw-r--r--  1 root root 1048 Jun 23 10:49 docker-compose.yml
-rw-r--r--  1 root root 2050 Jun 23 11:46 model.py
-rw-r--r--  1 root root   35 Jun 23 10:41 requirements.txt
drwx-----  2 root root 4096 Jun 23 11:46 templates
```

Рисунок 3 - Файлы проекта, размещённые на сервере

Листинг 4 - Результат выполнения команды **docker compose build**

```
root@olya-vds:~/currency-rate# docker compose build
WARN[0000] Docker Compose requires buildx plugin to be installed
Sending build context to Docker daemon  2.536kB
Step 1/8 : FROM python:3.13-slim
3.13-slim: Pulling from library/python
cf5110b4fa79: Pull complete
9970f4c20ff1: Pull complete
bae41854fae8: Pull complete
ce7c7b69704b: Download complete
cb6a9471781e: Download complete
Digest:
sha256:c33f0bc4364a6881bed1ec0cc2665e6c53c87a43e774aaeab88e6f17a
f105e4f
Status: Downloaded newer image for python:3.13-slim
---> c33f0bc4364a
Step 2/8 : WORKDIR /app
```

```
---> Running in 294ce687d4d1
---> Removed intermediate container 294ce687d4d1
---> ebfd734f7764
Step 3/8 : COPY requirements.txt .
---> d3e9bae379ba
Step 4/8 : RUN pip install --no-cache-dir -r requirements.txt
---> Running in 98clce3e7efc
Collecting flask (from -r requirements.txt (line 1))
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting sqlalchemy (from -r requirements.txt (line 2))
  Downloading sqlalchemy-2.0.51-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (9.5 kB)
Collecting requests (from -r requirements.txt (line 3))
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting gunicorn (from -r requirements.txt (line 4))
  Downloading gunicorn-26.0.0-py3-none-any.whl.metadata (5.4 kB)
Collecting blinker>=1.9.0 (from flask->-r requirements.txt (line 1))
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask->-r requirements.txt (line 1))
  Downloading click-8.4.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask->-r requirements.txt (line 1))
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask->-r requirements.txt (line 1))
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask->-r requirements.txt (line 1))
```

Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)

Collecting werkzeug>=3.1.0 (from flask->-r requirements.txt (line 1))

Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)

Collecting greenlet>=1 (from sqlalchemy->-r requirements.txt (line 2))

Downloading greenlet-3.5.2-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (3.8 kB)

Collecting typing-extensions>=4.6.0 (from sqlalchemy->-r requirements.txt (line 2))

Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)

Collecting charset_normalizer<4,>=2 (from requests->-r requirements.txt (line 3))

Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (40 kB)

Collecting idna<4,>=2.5 (from requests->-r requirements.txt (line 3))

Downloading idna-3.18-py3-none-any.whl.metadata (6.1 kB)

Collecting urllib3<3,>=1.26 (from requests->-r requirements.txt (line 3))

Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)

Collecting certifi>=2023.5.7 (from requests->-r requirements.txt (line 3))

Downloading certifi-2026.6.17-py3-none-any.whl.metadata (2.5 kB)

Collecting packaging (from gunicorn->-r requirements.txt (line 4))

Downloading packaging-26.2-py3-none-any.whl.metadata (3.5 kB)

Downloading flask-3.1.3-py3-none-any.whl (103 kB)

Downloading sqlalchemy-2.0.51-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (3.3 MB)

3.3/3.3 MB 84.5 MB/s 0:00:00

Downloading requests-2.34.2-py3-none-any.whl (73 kB)

Downloading charset_normalizer-3.4.7-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (215 kB)

Downloading idna-3.18-py3-none-any.whl (65 kB)

Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)

Downloading gunicorn-26.0.0-py3-none-any.whl (212 kB)

Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)

Downloading certifi-2026.6.17-py3-none-any.whl (133 kB)

Downloading click-8.4.1-py3-none-any.whl (116 kB)

Downloading greenlet-3.5.2-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (616 kB)

616.2/616.2 kB 459.2 MB/s 0:00:00

Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)

Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)

Downloading markupsafe-3.0.3-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)

Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)

Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)

Downloading packaging-26.2-py3-none-any.whl (100 kB)

Installing collected packages: urllib3, typing-extensions, packaging, markupsafe, itsdangerous, idna, greenlet, click, charset_normalizer, certifi, blinker, werkzeug, sqlalchemy, requests, jinja2, gunicorn, flask

Successfully installed blinker-1.9.0 certifi-2026.6.17 charset_normalizer-3.4.7 click-8.4.1 flask-3.1.3 greenlet-3.5.2 gunicorn-26.0.0 idna-3.18 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 packaging-26.2 requests-2.34.2 sqlalchemy-2.0.51 typing-extensions-4.15.0 urllib3-2.7.0 werkzeug-3.1.8

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>. Use the --root-user-action option if you know what you are doing and want to suppress this warning.

---> Removed intermediate container 98clce3e7efc

---> c45fbcaf5462

Step 5/8 : COPY . .

---> 13feae69c109

Step 6/8 : EXPOSE 5000

---> Running in 1265108bcc04

---> Removed intermediate container 1265108bcc04

---> 6b35ff3ed20f

Step 7/8 : CMD ["gunicorn", "-b", "0.0.0.0:5000", "app:app"]

---> Running in bb925e72b9d6

---> Removed intermediate container bb925e72b9d6

---> a3616d178c73

Step 8/8 : LABEL com.docker.compose.image.builder=classic

---> Running in 06613a3938fb

---> Removed intermediate container 06613a3938fb

---> 6450d9a58bcc

Successfully built 6450d9a58bcc

Successfully tagged currency-rate-app:latest

[+] build 1/1

✓	Image	currency-rate-app	Built
22.8s			

root@olya-vds:~/currency-rate#

Запуск контейнера приложения (Рисунок 4).

```
root@olya-vds:~/currency-rate# docker compose up -d
[+] up 2/2
✓Network currency-rate_default Created
✓Container currency-app Started
root@olya-vds:~/currency-rate# |
```

Рисунок 4 - Результат выполнения команды docker compose up -d

Работающий контейнер приложения (Рисунок 5).

```
root@olya-vds:~/currency-rate# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
3b9ce6a29308   currency-rate-app  "nginx -b 0.0.0.0..."  39 seconds ago Up 39 seconds  0.0.0.0:5003->5000/tcp, [::]:5003->5000/tcp  curren
ncy-app
```

Рисунок 5 - docker ps

Веб-приложение «Курсы валют ЦБ РФ», запущенное в контейнере Docker (Рисунок 6).

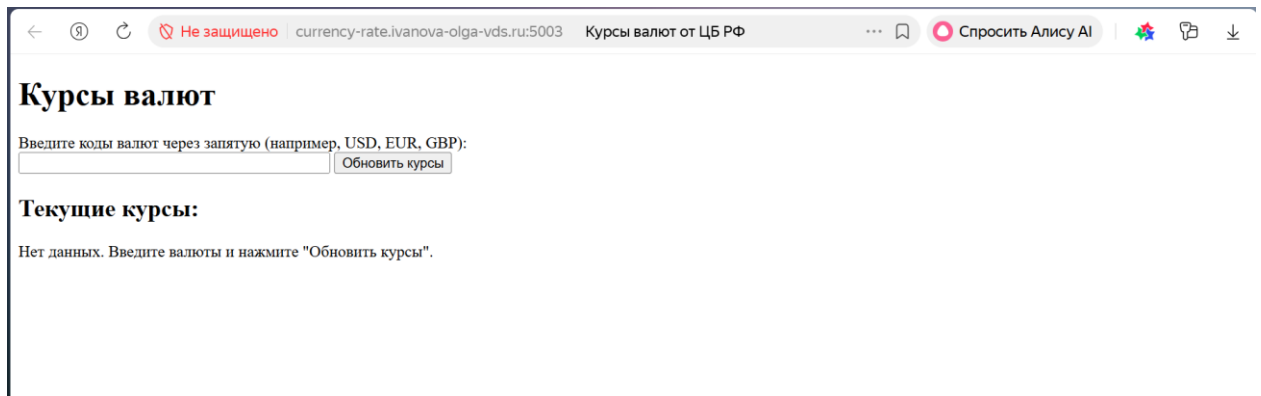


Рисунок 6 – Демонстрация работы сайта <http://currency-rate.ivanova-olga-vds.ru:5003>

Листинг 5 - Конфигурация Nginx для проксирования запросов к контейнеру приложения

```
server {

    server_name currency-rate.ivanova-olga-vds.ru;

    location / {

        proxy_pass http://127.0.0.1:5003;

        proxy_set_header Host $host;
```

```

        proxy_set_header X-Real-IP $remote_addr;

        proxy_set_header                                X-Forwarded-For
$proxy_add_x_forwarded_for;

        proxy_set_header X-Forwarded-Proto $scheme;

    }

}

```

Активация конфигурации виртуального хоста (Рисунок 7).

```

root@olya-vds:~/currency-rate# ln -s /etc/nginx/sites-available/currency-rate.ivanova-olga-vds.ru /etc/nginx/sites-enabled/
root@olya-vds:~/currency-rate# ls /etc/nginx/sites-enabled
currency-rate.ivanova-olga-vds.ru  default  lab.ivanova-olga-vds.ru  test.ivanova-olga-vds.ru

```

Рисунок 7

Проверка конфигурации Nginx (Рисунок 8).

```

root@olya-vds:~/currency-rate# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@olya-vds:~/currency-rate# |

```

Рисунок 8 - nginx -t

Перезапуск веб-сервера Nginx (Рисунок 9).

```

root@olya-vds:~/currency-rate# systemctl reload nginx
root@olya-vds:~/currency-rate# systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-16 06:26:58 UTC; 1 week 0 days ago
     Docs: man:nginx(8)
  Process: 67509 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 67514 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
  Process: 171470 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s reload (code=exited, status=0/SUCCESS)
 Main PID: 67538 (nginx)
    Tasks: 2 (limit: 1116)
   Memory: 4.9M (peak: 7.9M)
      CPU: 8.040s
   CGroup: /system.slice/nginx.service
           └─ 67538 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
              └─ 171473 "nginx: worker process"

Jun 23 12:09:28 olya-vds systemd[1]: Reloading nginx.service - A high performance web server and a reverse proxy server...
Jun 23 12:09:28 olya-vds nginx[171470]: 2026/06/23 12:09:28 [notice] 171470#171470: signal process started
Jun 23 12:09:28 olya-vds systemd[1]: Reloaded nginx.service - A high performance web server and a reverse proxy server.
Notice: journal has been rotated since unit was started, output may be incomplete.

```

Рисунок 9

Автоматический выпуск SSL-сертификата Let's Encrypt (Рисунок 10).

```
root@olya-vds:~/currency-rate# certbot --nginx -d currency-rate.ivanova-olga-vds.ru
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Requesting a certificate for currency-rate.ivanova-olga-vds.ru

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/currency-rate.ivanova-olga-vds.ru/fullchain.pem
Key is saved at: /etc/letsencrypt/live/currency-rate.ivanova-olga-vds.ru/privkey.pem
This certificate expires on 2026-09-21.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

Deploying certificate
Successfully deployed certificate for currency-rate.ivanova-olga-vds.ru to /etc/nginx/sites-enabled/currency-rate.ivanova-olga-vds.ru
Congratulations! You have successfully enabled HTTPS on https://currency-rate.ivanova-olga-vds.ru

-----
If you like Certbot, please consider supporting our work by:
* Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
* Donating to EFF: https://eff.org/donate-le
-----
root@olya-vds:~/currency-rate#
```

Рисунок 10 - Сертификат SSL

Работа веб-приложения по защищённому протоколу HTTPS (Рисунок 11).

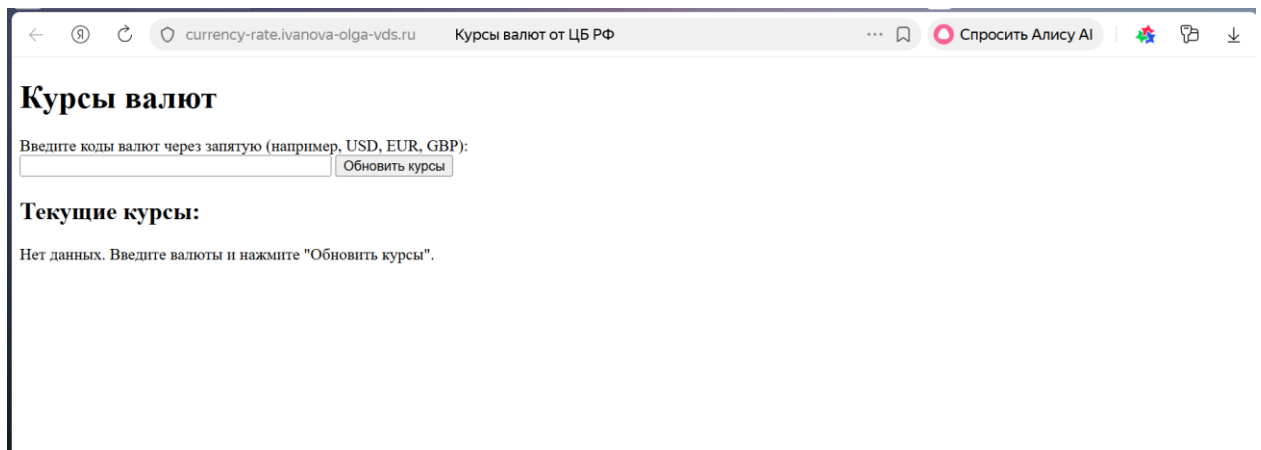


Рисунок 11 - Демонстрация работы сайта <https://currency-rate.ivanova-olga-vds.ru>

Служба автоматического обновления сертификатов Certbot (Рисунок 12).

```
root@olya-vds:~/currency-rate# systemctl status certbot.timer
● certbot.timer - Run certbot twice daily
   Loaded: loaded (/usr/lib/systemd/system/certbot.timer; enabled; preset: enabled)
   Active: active (waiting) since Wed 2026-06-10 14:30:27 UTC; 1 week 5 days ago
   Trigger: Tue 2026-06-23 16:24:44 UTC; 4h 9min left
   Triggers: ● certbot.service

Notice: journal has been rotated since unit was started, output may be incomplete.
```

Рисунок 12 – systemctl status certbot.timer

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была рассмотрена и реализована задача контейнеризации веб-приложения для отслеживания курсов валют Центрального банка Российской Федерации с использованием технологии Docker, а также организации защищённого доступа к приложению посредством HTTPS и автоматического выпуска SSL-сертификатов Let's Encrypt.

В рамках практической части была выполнена контейнеризация приложения с использованием Docker, а также разработана конфигурация Docker Compose, позволяющая запускать многокомпонентное приложение в единой среде. Дополнительно была настроена интеграция с веб-сервером Nginx, выполняющим функцию обратного прокси-сервера, обеспечивающего маршрутизацию запросов и доступ к приложению через доменное имя.

Также была реализована автоматизация выпуска и обновления SSL-сертификатов с использованием Let's Encrypt, что позволило обеспечить защищённую передачу данных по протоколу HTTPS без необходимости ручного администрирования сертификатов.

Таким образом, все поставленные цели курсовой работы были достигнуты, а полученный опыт демонстрирует практическую применимость технологий контейнеризации, автоматизации развертывания и обеспечения безопасности веб-приложений.

СПИСОК ЛИТЕРАТУРЫ

1. Щуцкий, Д. Гайд по деплою web-приложений для новичков. Часть 1. Shared-хостинг [Электронный ресурс]. – URL: <https://habr.com/ru/articles/820473/> (дата обращения: 15.06.2026).
2. Rohith, S.V. Docker or Virtual Machines: What's the Right Choice? [Электронный ресурс]. – URL: <https://www.opensourceforu.com/2025/11/docker-or-virtual-machines-whats-the-right-choice/> (дата обращения: 15.06.2026).
3. Самойленко, А. Открытая платформа виртуализации VirtualBox [Электронный ресурс]. – URL: <https://www.ixbt.com/cm/virtualization-virtualbox.shtml> (дата обращения: 15.06.2026).
4. Мосееенкова, К. Погружение в контейнеризацию [Электронный ресурс]. – URL: <https://habr.com/ru/companies/otus/articles/767884/> (дата обращения: 16.06.2026).
5. Хомутов, А. Docker для начинающих: что это такое и как пользоваться [Электронный ресурс]. – URL: <https://habr.com/ru/companies/netologyru/articles/967546/> (дата обращения: 16.06.2026).
6. Docker Inc. Docker Compose [Электронный ресурс]. – URL: <https://docs.docker.com/compose> (дата обращения: 16.06.2026).
7. Docker Inc. How Compose works [Электронный ресурс]. – URL: <https://docs.docker.com/compose/intro/compose-application-model> (дата обращения: 17.06.2026).
8. Веб-сервер Nginx и что о нём должен знать каждый разработчик [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/chto-takoe-nginx/> (дата обращения: 17.06.2026).
9. F5, Inc. NGINX Reverse Proxy [Электронный ресурс] // NGINX Documentation. – URL: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/> (дата обращения: 17.06.2026).

- 10.Let's Encrypt [Электронный ресурс] // Википедия. – URL: https://ru.wikipedia.org/wiki/Let's_Encrypt (дата обращения: 19.06.2026).
- 11.Что такое Let's Encrypt и как настроить бесплатный SSL-сертификат? [Электронный ресурс] // Hostragons. – URL: https://www.hostragons.com/ru/блог/как-установить-бесплатный-ssl-сертификат/#Обновления_сертификатов (дата обращения: 20.06.2026).